NIXOS PORTABILITY AUDIT · 2026-07-13

Nix configuration portability and immediate-usability review

An evidence-backed assessment of firmware, drivers, Wi-Fi, installation, desktop behavior, theming, cross-platform support, and operational proof.

60 supplied sources | 141 observations | 58 canonical claims | 6 exported configurations

Research date: 2026-07-13
Production revision reviewed: `e9f78180748f1feb428ffb20f9d932c5d9918a48`
Root Nixpkgs revision: `d407951447dcd00442e97087bf374aad70c04cea`
Scope: the current repository, all 60 supplied source entries, exact pinned upstream code, six declared configurations, generated closures/units, bounded VM and fault-injection tests, and adversarial review. No production configuration was changed.

Important

The repository is a promising **mainstream x86 workstation baseline**, not an "install unchanged on any machine and everything works immediately" system. Literal universal hardware support is not a technically defensible promise. The achievable standard is: **one repository, shared policies, bounded compatibility-class profiles, named host/storage leaves, fail-closed install tooling, and fresh physical acceptance evidence for every supported host**.

How to read the evidence

Every conclusion belongs to one of five states. The label is written out so color is never the only signal.

P0 · BLOCKS RELEASE

Act before another unattended install

Credential response, remote-install trust, destructive disk identity, the red production fixture, and broken desktop authority require repair and fresh proof.

P1 · RELIABILITY DEBT

Qualify each supported host

Firmware, microcode, Wi-Fi, boot capacity, Niri runtime behavior, theming, and platform ownership need bounded profiles and acceptance records.

VERIFIED · KEEP

Preserve working foundations

Six outputs evaluate, the isolated UEFI/Disko path boots, shared Kitty policy reaches macOS, and bootstrap secret lifecycle tests are strong.

DEFERRED · NOT GREEN

Require physical or external evidence

Real WLAN devices, native Mac activation, Secure Boot, offline installation, destructive-machine recovery, and provider-side rotation remain unverified.

REFUTED · DO NOT REPEAT

Discard claims the evidence disproved

Adversarial checks rejected false absolutes about universal closures, duplicate services, ESP capacity, microcode, rollback, macOS theming, and active age secrets.

Executive verdict

What is already good

- All six declared configurations evaluate to store paths, and the exact x86 NixOS closure was realized. The x86 closure contains a broad generic kernel/initrd baseline, redistributable firmware, `wireless-regdb`, Mesa, Bluetooth, PipeWire, polkit, NetworkManager, and its intentionally D-Bus-controlled wpa_supplicant backend. This is credible static evidence for a bounded mainstream-PC class; it is not physical hardware proof.
- After isolating only a missing test fixture secret, the exact Disko layout, systemd-boot installation, OVMF reboot, and `local-fs.target` passed in a VM.
- The Niri portal selection is broadly coherent for screenshot/screencast, chooser/settings, and Secret. The integrated NixOS generation contains one Noctalia unit—not duplicate NixOS and Home Manager units.
- Kitty palette/font declarations reach all six configurations, macOS included. The absence of native CoreText/Kitty rendering evidence is the gap, not the absence of declarations.
- Bootstrap password staging, shape, ownership, mutation, cleanup, and lifecycle tests passed. A disposable age test kept plaintext out of the store and logs.
- The Den/aspect composition is stronger than most reference dotfiles. It does not need another framework merely to become portable; it needs explicit host leaves and exported evidence.

What blocks the desired promise now

1. **Incident action:** a tracked third-party API credential exists in the current public tree, reachable history, and a Nix store source. Its value was deliberately neither reproduced nor tested. Rotate/revoke it externally before repository cleanup. This is an immediate remediation priority, while the evidenced impact is a read-only API

confidentiality exposure—not proven account takeover. Follow GitHub's secret-removal guidance (https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository).

2. **Remote install trust:** pinned/current nixos-anywhere puts `UserKnownHostsFile=/dev/null` and `StrictHostKeyChecking=no` before appended caller options, and OpenSSH uses the first value. The helper therefore cannot be repaired by simply appending strict options. This is critical when a destructive remote install traverses an unauthenticated network, but not when using a verified local ISO/console or independently authenticated tunnel. See the exact option order (https://github.com/nix-community/nixos-anywhere/blob/4dfb813db065afb0aba1f61658ef77993d382db1/src/nixos-anywhere.sh#L63-L70) and upstream issue #552 (https://github.com/nix-community/nixos-anywhere/issues/552).
3. **Kexec trust:** the default kexec path downloads and executes a mutable release asset as root without binding it to a digest or signature. Pinning nixos-anywhere's source does not pin those bytes. This risk applies only when that kexec path runs; a recognized checksum-verified NixOS installer or install-only flow bypasses it. See the exact downloader (https://github.com/nix-community/nixos-anywhere/blob/4dfb813db065afb0aba1f61658ef77993d382db1/src/nixos-anywhere.sh#L709-L816).
4. **Disk identity:** the literal `/dev/%DISK%` currently fails closed, but replacing it with the wrong *valid* disk is destructive. The Disko VM test rewrites the device and cannot prove the physical selector. The current leaf is at `modules/nixos/disk-config.nix:7`; Disko's test rewrite is visible in upstream source (https://github.com/nix-community/disko/blob/ff8702b4de27f72b4c78573dfb89ec74e36abdf1/lib/tests.nix#L11-L69).
5. **A real red release test is hidden:** the production-derived x86 Disko `installTest` is red because its fixture does not stage the mandatory bootstrap hash. Test-only isolation proves that the disk/boot path itself can boot; it does **not** prove the real extra-files choreography. The red test is not exported in `flake checks`.
6. **Niri's manual lock is broken:** the generated binding invokes `noctalia msg screen-lock`, which the exact Noctalia pin does not register. The supported ext-session-lock command is `noctalia msg session lock`. See `modules/linux/config/niri/config.kdl:58` and the pinned Noctalia IPC source (https://github.com/noctalia-dev/noctalia/blob/d8d8ed597e6a3b5a6846c0b474e6f67f573dc630/src/shell/session/session_ipc.cpp#L38-L82).
7. **Desktop authority is nondeterministic:** Noctalia is target-started while Mako and Dunst remain activatable candidates for `org.freedesktop.Notifications`. Picom, xautolock, xss-lock/i3lock, and Polybar are unconditionally scheduled into the Niri generation. This does not prove all are continuously healthy, but it does prove the Wayland role still owns obsolete X11 policy.
8. **Platform false greens:** global `allowUnsupportedSystem = true` hides real selections—Google Chrome in the AArch64 Linux graph and Icarus Verilog in the Intel-Darwin graph. `allowBroken = true` is currently latent, not proven to be masking a selected broken package. Both global escapes are in `lib/nixpkgs.nix:43-46`.

9. **Intel Darwin is outside the rolling support contract:** the repository is in the Nixpkgs 26.11 era, while 26.05 is the final supported `x86_64-darwin` release. Evaluation success cannot override the Nixpkgs support policy (https://nixos.org/manual/nixpkgs/unstable/release-notes#x86_64-darwin-26.05).
10. **Installed SSH is broader than intended:** password and keyboard-interactive SSH remain allowed, root and `mei` accept the same key, and consuming the bootstrap verifier does not rotate or lock the real shadow password. The shared-key marginal escalation is lower because the user is already root-equivalent through Docker/Nix/admin boundaries; direct root and password SSH should still be removed.

Recommended next move

Do **not** run another unattended remote install from the current production tree. First close the P0 incident, install-trust, disk-manifest, test-fixture, session-lock, notification-owner, platform-policy, and SSH issues. Then enroll the laptop as a named host from a checksum-verified local installer, with an attended disk confirmation, local console, and tested USB-Ethernet/tether fallback. Only after the physical checklist passes should that exact host be called ready.

What “works on any machine” can truthfully mean

It cannot mean one immutable closure, one bootloader, one disk layout, one GPU policy, or one firmware set works on every x86 PC, ARM board, Mac, VM, WSL instance, and server. Those targets differ before NixOS even starts: firmware bitness, UEFI versus BIOS/U-Boot, device tree versus ACPI, storage topology, kernel drivers, legally redistributable firmware, GPU topology, power quirks, and recovery interfaces.

The defensible contract is:

1. **Shared policy:** Nix behavior, base packages, users, shell, security defaults, common Home Manager configuration, and platform-neutral theme intent.
2. **Platform class:** NixOS, nix-darwin, standalone Home Manager, WSL, server, VM/cloud, or installer/recovery image. WSL is not a NixOS boot/disk/hardware target; Windows owns those layers. macOS is not NixOS with a different system string.
3. **Role:** workstation, laptop, headless server, builder, media server, or recovery image. A server should not inherit Niri merely because composition is convenient.
4. **Compatibility class:** for example, mainstream 64-bit UEFI x86 PC with Secure Boot off, one AHCI/NVMe disk, GPT/ext4, in-kernel GPU/network drivers, and redistributable firmware.

5. **Named host leaf:** sanitized hardware evidence, exact architecture/boot mode, stable storage identity, firmware/driver exceptions, model quirks, network fallback, power policy, and tested support claims.
6. **Per-host acceptance record:** commit, closure path, kernel, firmware, PCI/USB IDs, boot and rollback results, Wi-Fi/audio/GPU/suspend/session tests, date, and expiry trigger. Hardware, kernel, firmware, bootloader, or major desktop changes invalidate the relevant evidence.

A generic closure *can* span a bounded compatibility class; the earlier absolute claim that it cannot span diverse hardware was refuted. The problem is not genericity itself. The problem is advertising unbounded support without naming the boundary or testing representative physical members. The official NixOS manual's all-hardware profile (https://nixos.org/manual/nixos/stable/#sec-profile-all-hardware) is primarily an installer/recovery baseline, not a universal installed-host answer, while NixOS Facter (https://nixos.org/manual/nixos/stable/#sec-nixos-facter) and `nixos-generate-config` provide per-target evidence rather than a reusable report for every machine.

Current readiness and six-configuration matrix

Readiness is reported as evidence states rather than an invented numerical score. The dimensions are exact evaluation, native realization, installation/activation, functional behavior, and physical qualification. A dimension is described only by evidence actually retained in this bundle; no partial points or implied probabilities are assigned. Any open P0 blocks a release claim regardless of positive evidence elsewhere.

Readiness evidence for all six exported configurations

Configuration	Evaluation	Native realization	Install/activation	Functional/physical evidence	Current verdict
`nixosConfigurations.x86_64-linux`	Exact output evaluates	Exact x86 closure realized	Isolated Disko/OVMF boots; production-derived fixture red; no physical install	Broad static baseline; no physical support class qualified	**P0-blocked baseline**, not machine-ready.
`nixosConfigurations.aarch64-linux`	Evaluates only with unsupported-package escape	No native ARM realization	None	No named SystemReady machine/board or physical behavior	**Not ready and overclaimed.**
`homeConfigurations.standalone-linux`	Evaluates for fixed `mei` identity	Store path evaluation only; no representative native activation proof	None	User-profile scope cannot own host firmware/drivers/network/display-manager services	**Template for `/home/mei`, not generic.**
`homeConfigurations.standalone-linux-aarch64`	Evaluates for fixed `mei` identity	No native ARM realization/activation	None	Same Home Manager boundary plus unknown ARM host	**Not ready beyond evaluation.**

Configuration	Evaluation	Native realization	Install/activation	Functional/physical evidence	Current verdict
`darwinConfigurations.aarch64-darwin`	Evaluates for existing `mei` at `/Users/mei`	No native build	None	Kitty/font declarations only; Aqua/TCC/CoreText/apps/rollback unobserved	**Evaluation-ready only; native proof required.**
`darwinConfigurations.x86_64-darwin`	Evaluates under permissive policy	No supported native build	None	Rolling graph is beyond Intel-Darwin support; Icarus Verilog unavailable	**Unsupported as pinned; retire or isolate.**

Truthful support classes today

Truthful hardware support classes and current evidence boundaries

Class	Status	Boundary
Mainstream x86_64 desktop/laptop; 64-bit UEFI; Secure Boot off; one SATA-AHCI/NVMe disk; GPT/ext4; ordinary USB input	**Conditional baseline**	Stable disk identity, ESP/rollback correction, P0 fixes, and physical qualification still required.
Intel/AMD integrated graphics using in-kernel Mesa	**Conditional baseline**	External display, media acceleration, firmware, suspend, brightness, and Niri need physical proof.
In-kernel Wi-Fi driver using included redistributable firmware	**Plausible local setup**	Controller-specific scan/association not proven; no automatic credential handoff; remote kexec Wi-Fi is not a supported contract.
x86 QEMU/OVMF synthetic disk	**Partially VM-tested**	Test-only secret isolation booted; production fixture and physical selector did not pass.
AArch64 SystemReady/UEFI/ACPI	**Plausible unverified template**	No named target, native realization, or boot.
NVIDIA discrete/hybrid, Broadcom/out-of-tree WLAN, Surface/T2/vendor quirks	**Unsupported or unverified exception class**	Needs explicit driver/firmware/profile choice and physical evidence.
Legacy BIOS, 32-bit EFI, Secure-Boot-required x86	**Unsupported**	Requires separate GRUB/EFI-bitness/signing/enrollment/recovery policy.
ARM SBC/U-Boot/device-tree, Raspberry Pi image, Apple Silicon/T2	**Unsupported/unmodeled**	Requires board-specific firmware/kernel/device-tree/image/storage policy.
RAID/LVM/LUKS/ZFS, multiple disks, swap/hibernation	**Unsupported by current storage leaf**	Current Disko is one GPT disk, 100 MiB ESP, ext4 root, no encryption or swap.
Hyper-V/VMware/Virtio/Xen/cloud	**Unverified role class**	Guest agents, root initrd, console, firmware, and image layout are not tested together.
WSL/headless server	**Separate platform/role**	WSL has no native NixOS boot or Disko contract; servers must not inherit the workstation role by default.

Prioritized remediation plan

P0 — before another unattended install or readiness claim

Prioritized P0 remediation actions and required closure evidence

Order	Action	Proof required before closure
1	Externally rotate/revoke the exposed provider credential and review account activity; never test the old value. Then remove current copies and add a pinned, redacted current-tree/full-history scanner.	Provider-side revocation record; clean/leak scanner controls; zero value bytes in reports and current tree. History rewrite is a separate coordinated decision.
2	Avoid or patch nixos-anywhere's insecure SSH option ordering. For remote work, bind a private known-hosts file to a console-verified ED25519 fingerprint before any upload or disk action.	Known key succeeds; wrong/unknown/changed key fails before key upload, closure copy, or Disko.
3	Avoid default mutable kexec or replace it with a locally realized revision/NAR/digest-bound artifact. A checksum-verified direct ISO is the minimal attended path.	Good digest/signature succeeds; one-byte mutation fails before upload/execution; stage fingerprints remain continuous.
4	Replace `%DISK%` text substitution with a named-host storage leaf and no-write manifest gate checking stable by-id, resolved node, model, serial/WWN, size, transport, removable flag, descendants, architecture, and boot mode.	Any mismatch blocks before destructive phases; mounted source/installer media cannot be selected.
5	Make the Disko install test self-contained with dummy-secret missing/wrong/correct controls, then export it as a required check.	Production-derived test passes only with correct fixture; negative controls fail at the intended boundary.
6	Change the Niri binding to `noctalia msg session lock`; remove xautolock/xss-lock/i3lock from the Wayland role.	Exact command compatibility check plus cold-login, every-output, idle, suspend/resume, and hotplug lock tests.
7	Select one notification owner—Noctalia is the design-consistent choice—and remove Mako/Dunst activators from that role.	Cold login and restart show exactly one D-Bus owner without activation races.
8	Turn off global `allowUnsupportedSystem` and `allowBroken`; add only narrow, documented, expiring exceptions. Decide whether Intel Darwin is retired or frozen on supported Nixpkgs 26.05.	Strict policy evaluates every retained output natively; no global escape remains.
9	Disable installed root, password, and keyboard-interactive SSH; retain and test a separate recovery route. Require bootstrap password change or deliberate lock.	`sshd -T`, positive user-key/sudo tests, negative root/password tests, and `passwd -S` / `chage -l` lifecycle evidence.

P1 — day-one machine and desktop reliability

1. Split shared policy, role, hardware, storage/boot, and named host identity. Enroll the real laptop instead of treating `x86_64-linux` as a host name.
2. Keep redistributable firmware and `wireless-regdb`; enable the correct Intel or AMD microcode per host. Add proprietary firmware/driver exceptions only for a recorded device ID and tested kernel.

3. Move NetworkManager ownership from the Niri implementation module into a workstation-network role. If Facter is imported, set `hardware.facter.detected.dhcp.enable = false` and assert dhcpcd is absent.
4. Supply a safe first-connect path: local `nmtui` with console access by default, or an encrypted root-only NetworkManager keyfile over an already authenticated installer path. Test USB Ethernet or phone tethering.
5. Replace the 100 MiB ESP/42-entry mismatch. A normal policy such as 1 GiB and 10 retained entries is defensible only with a measured free-byte preflight; retaining 42 fully unique current payloads needs at least a roughly 3 GiB XBOOTLDR policy, not a hopeful entry count.
6. Define rollback as an invariant spanning Nix roots, ESP bytes, known-good boot entries, lock file, installer artifact, and mutable-data backup. Do not let seven-day cleanup erase the only proven generation.
7. Remove or explicitly X11-scope Picom and Polybar. Choose one wallpaper owner. Install OBS Studio or remove its launcher, then test a real PipeWire portal capture.
8. Choose one GTK authority and install the assets it names. `Sweet-cursors`, Fira Sans, and Sarasa UI SC are currently requested but absent. Qt is redundant rather than proven contradictory; simplify it only after runtime comparison.
9. Replace fixed `mei` identity assumptions with one typed identity source and test two users/homes. Select one Darwin GUI application projection owner.
10. Repair `sync-secrets` so it accepts a canonical writable runtime worktree; never make plaintext visible to Nix evaluation. Use per-host agenix identities only when secrets are actually activated.
11. Build a true offline host kit: clean source snapshot, lock/archived inputs, full named-host closure, pinned installer closure, firmware, hashes, and a disconnected cold-store test. A Git checkout on Ventoy is not enough.
12. Export existing semantic tests, both Disko tests, native builds, Linux VM tests, strict installer tests, and release/physical gates through a proportional CI hierarchy.
13. Make Emacs deterministic: HTTPS-only archives, content-bound or Nix-owned packages, and clean-home offline startup tests on native Linux and Darwin.
14. Replace or bind the README's moving Determinate `curl | sh` bootstrap to a reviewed version/digest/signature, with a verified offline alternative.
15. Preserve standalone Linux as backup-safe incremental adoption: generic public outputs, no private bootstrap dependency, and host ownership of gaming/GPU/system integration until an explicit NixOS migration.
16. Keep Darwin Homebrew-free unless a new explicit decision supersedes the policy. Assert no casks, Homebrew module, or `/opt/homebrew` fallback.
17. Preserve the public `x86_64-linux` / `aarch64-linux` aliases, paired Bash/Nushell install examples, and Niri-only login while adding named hosts.
18. Keep external-display privilege boundaries explicit: NixOS owns I2C declaratively; standalone uses an attended helper. Test DDC physically.

19. Move timezone and XKB layout from the shared NixOS baseline into typed host/user/location policy and evaluate at least two differing fixtures.

P2 — lifecycle and optional capabilities

- Decide fwupd, rtkit, automatic upgrade, automatic GC, printing, hibernation, and power policy per role/host. Enabling each everywhere would be another universality error. `security.rtkit.enable = true` is the documented PipeWire baseline, but it still needs an audio/session test.
- Model Ollama/llama.cpp acceleration as a host capability with CPU fallback. CUDA, ROCm, Vulkan, and Metal labels are not interchangeable. Benchmark on the physical GPU before calling the backend supported.
- Decide a shared editor policy before adding VS Code. Zed is broadly packaged, but its configuration currently belongs only to standalone Linux. Extension provenance and platform availability should be explicit.
- Record source, revision, license, attribution, and modification status for wallpapers/logos/icon subsets. Replace uncertain or restrictive assets where distribution obligations are undesirable.
- Add formatter/dev-interface polish only after the current red release test, installer trust, credential incident, and platform escapes are resolved.

Firmware, drivers, and hardware

Current facts

The exact x86 closure contains a broad driver and redistributable-firmware baseline. This refutes “the config has no Wi-Fi support” and “missing microcode prevents generic boot.” It does **not** establish that the unidentified laptop's controller has a compatible driver or firmware. Both Intel and AMD microcode options are false, `hardware.enableAllFirmware` is false, and nonredistributable packages such as bounded Broadcom/B43 cases are absent.

Keep `hardware.enableRedistributableFirmware = true` as the shared physical-workstation baseline. Do not respond by globally enabling everything:

- `hardware.enableAllHardware` expands the initrd significantly and is useful for installer/recovery media, but cannot solve board boot chains, NVIDIA PRIME topology, firmware licenses, or model quirks.
- `hardware.enableAllFirmware` is a bounded package set, not “all drivers.” It still cannot create a missing kernel driver or validate a device/kernel pair.

- `nixos-hardware` should be imported only after matching manufacturer and exact model and reviewing the module. A nearby laptop with the same CPU is not the same target.
- Facter should be a sanitized per-host assurance input. Raw reports can contain persistent identifiers and become store-visible when used as Nix input.
- Generic Mesa belongs in shared policy; NVIDIA branch/open-module choice, PRIME bus IDs, CUDA, legacy AMD, special media acceleration, and hybrid power are host capabilities.
- Power tuning is host-specific. TLP and aggressive PCI/USB power-saving can cause freezes or suspend/resume regressions. Prefer a measured host policy, not stacked power daemons.

Host architecture to implement

```
shared/base
  ├── nix, users, common CLI, security defaults
platform/nixos
  ├── Linux kernel/service policy
role/workstation
  ├── Niri, NetworkManager, PipeWire, Bluetooth, portals
hardware/<named-host>
  ├── sanitized Facter or reviewed generated config
  ├── exact nixos-hardware profile if applicable
  ├── microcode/GPU/WLAN/firmware/power exceptions
storage/<named-host>
  ├── stable by-id, boot mode, ESP/XBOOTLDR, filesystem, swap/LUKS
host/<named-host>
  └── composes role + hardware + storage + identity + evidence metadata
```

The NixOS installation manual (https://nixos.org/manual/nixos/stable/#sec-installation-manual) and NixOS manual (https://nixos.org/manual/nixos/stable/) remain the authority for generated hardware configuration, networking, boot, and tests. Community repositories are pattern catalogs, not target hardware evidence.

Wi-Fi and first-boot connectivity

Correct diagnosis of the observed laptop

Seeing only `lo` and `docker0` is **not** explained by a missing SSID/profile. A profile affects association after a netdev exists. It also is not explained by two competing managers: at this pin, NetworkManager intentionally controls wpa_supplicant over D-Bus. The missing interface still requires physical controller/driver/firmware/rfkill/kernel diagnosis.

The installed stack is coherent and lives in the closure; the remote nixos-anywhere/kexec contract nevertheless does not promise Wi-Fi. For the next install, use one of these ordered choices:

1. **Safest:** boot a verified installer, use local console and wired/USB Ethernet or tethering, then run `nmtui` after first boot. The root-only keyfile stays in `/etc/NetworkManager/system-connections` and never enters Nix.
2. **Seamless attended handoff:** connect with `nmtui` in the ISO, identify the active system keyfile without printing secrets, copy it through an authenticated channel into tmpfs, validate regular-file/owner/mode/SSID/UUID metadata, and stage it mode `0600`, owner `0:0`. Destroy staging via trap.
3. **Declarative long-term:** use `networking.networkmanager.ensureProfiles.environmentFiles` with a runtime agenix path, not a literal PSK or `builtins.readFile`. The exact module creates the runtime profile with restrictive permissions; see the pinned NetworkManager module (https://github.com/NixOS/nixpkgs/blob/d407951447dcd00442e97087bf374aad70c04cea/nixos/modules/services/networking/networkmanager.nix#L2853-L2897) and NetworkManager secret flags (https://networkmanager.dev/docs/api/latest/nm-settings-nmcli.html#secrets).

If Facter is added to this NetworkManager role, exact-pin synthetic evaluation shows its detected-interface DHCP path starts dhcpcd unless explicitly disabled. That is a **prospective enrollment collision**, not a current bug because the current outputs import no Facter report.

Boot, disk layout, ESP capacity, and recovery

Current boot boundary

The current configuration assumes 64-bit UEFI, systemd-boot, writable NVRAM, GPT, one disk, a 100 MiB FAT ESP, and ext4 root. The boot configuration is in `modules/nixos/system.nix:8-20`. It does not model legacy BIOS, 32-bit EFI, Secure

Boot, board-specific ARM boot, multiple disks, LUKS, RAID/LVM/ZFS, swap, or hibernation.

Secure Boot is absent, not merely untested: the active artifacts have empty PE Security Directories and no Lanzaboote policy. Mark it unsupported until a separate signing, key-enrollment, removable fallback, and recovery design passes on sacrificial firmware.

Measured ESP result

The 100 MiB FAT image had 104,634,368 usable bytes. The exact current kernel+initrd payload was 42,428,580 bytes. Two fully unique payloads fit; the third failed. A configuration entry limit is not a byte budget. The exact systemd-boot updater garbage-collects before writes and atomically replaces individual files, not a whole generation set. In synthetic ENOSPC injection, the selected old generation survived, one unselected generation did not, and an orphan temporary file could remain.

Use a dynamic preflight based on actual boot artifacts and preserve a selected last-known-good generation. The former suggestion that 2 GiB safely retains 42 full current payloads with 20% headroom was corrected: exact occupancy is 82.98%. A normal 1 GiB/10 policy is a reasonable starting point only with a byte gate; use at least roughly 3 GiB XBOOTLDR if 42 fully unique payloads are truly a requirement.

Disk-selection invariant

Before Disko performs any write, an attended or signed manifest must bind:

- named host and architecture;
- UEFI/BIOS/Secure Boot state;
- stable `/dev/disk/by-id/...` selector and its resolved node;
- model, serial/WWN where available, capacity, transport, removable bit;
- entire descendant/mount ancestry;
- exclusion of the running installer USB and every mounted valuable filesystem;
- expected partition plan and acceptable size tolerance.

The literal `%DISK%` being fail-closed is a useful negative property, but it is not identity binding. The VM's `/dev/vdb` rewrite is also not evidence about the physical disk.

Rollback invariant

Seven-day generation deletion and 42 boot entries do not define a rollback window. Before GC, preserve the last release lock, closure, installer, known-good boot generation, and mutable-data backup. Test `test` → health check → `boot` → reboot → previous generation → `switch`, then GC and repeat. Nix generations do not roll back mutable `/var`, databases, partitions, firmware/NVRAM, or user data; those require separate restore tests. Official Nix documentation warns that deleting generations destroys rollback ability (`nix-collect-garbage` (https://nix.dev/manual/nix/2.34/command-ref/nix-collect-garbage)).

Installation, SSH trust, secrets, and supply chain

Minimal defensible attended install

1. Boot a checksum-verified official or custom NixOS ISO at the local console. Prefer direct console or physically controlled Ethernet; this avoids default kexec.
2. Record the installer's ED25519 host fingerprint at the console. If remotely invoking a patched helper, use a private known-hosts file and prove a wrong fingerprint stops before any upload.
3. Select a named host leaf and run the no-write host/disk manifest check.
4. Keep wipe confirmation attended. Re-check identity immediately before Disko.
5. Include recovery tools (`pciutils`, `usbutils`, `iw`, `rfkill`, `ethtool`, `nmcli`) and a tested USB-Ethernet/tether escape hatch.
6. Use `--copy-host-keys` only after authenticating the initial key. It is a continuity mechanism, not a trust root. Compare the staged final fingerprint before reboot.
7. Attend first boot; verify hardware, Wi-Fi, SSH policy, password rotation, sudo, current boot and rollback before declaring enrollment complete.

For repeated unattended or hostile-network installs, add the Wave 2 signed, expiring per-host manifest and content-bound local kexec artifact. The local signature/digest harness passed all six good/tampered/wrong-key controls, but independent builder reproducibility, signer custody, and physical stage transitions remain unverified.

The standalone Home Manager path has a separate unresolved bootstrap boundary: the README pipes a moving Determinate installer response into a shell. Transport encryption is not content identity. Replace that instruction with reviewed, version/digest/signature-bound bytes or a verified offline route, and retain the reviewed installer artifact alongside the lock and rollback kit.

Standalone Linux must remain an **incremental, backup-safe adoption path** for existing Arch/CachyOS systems rather than silently becoming a full NixOS migration. Preserve generic public outputs, `hm-backup` takeover, bootstrap without private flake credentials, and host ownership of gaming/GPU/system integration until the user explicitly changes that boundary.

Secrets policy

- Rotate/revoke the exposed provider credential outside the repository. Do not validate the old value. Remove current copies only after rotation; then decide whether a coordinated history rewrite is worth forcing every collaborator to re-clone.

- `age.secrets` is currently empty. There is no current agenix plaintext activation to remediate. Before enabling it, use a dedicated per-host identity plus an offline recovery recipient; do not use the everyday login key as the long-term secret identity.
- Keep encrypted material tracked; decrypt to `/run/agenix` or another runtime path with restrictive owner/mode. Never interpolate plaintext into a Nix option, generated unit, environment, command argument, `writeText`, `home.file.source`, flake input, or `builtins.readFile`.
- The current `sync-secrets` writes toward `${self}`, which is an immutable Nix store source. Its failure is functional, not a new publication incident. Its repaired interface must accept an explicit canonical writable worktree and fail closed on containment checks.
- Consuming `/var/lib/nixos-bootstrap/mei-password.hash` removes a verifier copy, not the actual `/etc/shadow` password. Require first-use change (`chage -d 0`) or deliberate lock after verified key/sudo recovery.

SSH final policy

The normal user already has high-trust boundaries through Docker and trusted Nix access. That does not justify direct root login. Use normal-user keys, explicit sudo, `PasswordAuthentication no`, `KbdInteractiveAuthentication no`, and `PermitRootLogin no`, with a separately documented offline recovery route. Validate the generated daemon with `sshd -T`, not only Nix option values.

Niri, Noctalia, notifications, portals, and applications

Lock authority

The exact manual lock command is deterministically wrong. Replace it with `noctalia msg session lock`, then make the ext-session-lock owner the only lock authority. Remove xautolock/xss-lock/i3lock from the Niri role. KDL syntax validation cannot catch an invalid external subcommand, so add an exact-pin semantic check and physical multi-output tests.

Notifications

The generated graph offers Noctalia, Mako, and Dunst as candidates for the same Freedesktop singleton. An isolated activation selected Mako; a cold installed login might select another owner depending on timing. The accurate statement is not “three daemons definitely run simultaneously,” but “ownership is ordering-dependent.” Select one owner and remove the other activators.

X11 residue and wallpaper authority

Picom, xautolock, xss-lock/i3lock, and Polybar are scheduled into the Niri generation. Remove the lock stack as P0; remove or explicitly X11-scope Picom and Polybar as P1 hygiene. Both awww and Noctalia are wallpaper-capable, but no tracked `awww img` command proves a visible conflict. Treat it as ambiguous authority and needless resource use, not a demonstrated broken wallpaper.

The installed-machine contract intentionally exposes **Niri only**. Retain that assertion while deleting obsolete BSPWM/X11 residue; do not restore a second login session as a fallback. Named-host refactoring must keep the existing architecture-named public outputs as compatibility aliases, and installer documentation must continue to pair shell-sensitive Bash and Nushell commands.

Portals

The exact split is sensible for Niri ScreenCast/Screenshot, chooser/settings, and Secret. Niri does not implement GNOME Mutter's RemoteDesktop input-injection interface, so do not advertise full remote-control parity merely because screen capture works. Real consent dialogs, window/monitor selection, audio, sandboxed client behavior, and resume/hotplug remain physical session tests.

OBS and stale settings

The generated OBS wrapper/desktop entry exits 127 in the clean realized profile because OBS Studio is absent. Install it or remove the launcher, then test actual PipeWire capture. Noctalia validation returns success but reports stale keys; runtime-state merging is intentional upstream policy, not inherently a defect. Remove stale schema keys and decide whether declarative intent or mutable user state owns each setting.

Theming, Kitty, fonts, icons, wallpapers, and macOS

Static parity that is already present

Kitty palette/font declarations and Fira-named font packages reach all six Home Manager configurations. This directly refutes the suspicion that macOS was excluded. It proves configuration projection, not that CoreText registered the font or Kitty rendered identically on a real Mac.

Linux theme defects

- GTK has multiple writers: structured settings, forced `GTK_THEME`, GSettings, and copied theme files. Choose one authority and test GTK 3/4/libadwaita behavior.
- `Sweet-cursors`, Fira Sans, and Sarasa UI SC are named in current configs but absent from the realized trees. Either package those exact assets or select installed families. See the requested cursor/font values in `modules/linux/home-manager.nix:283-286` and Mako's font in `modules/standalone-linux/config/mako/config:3`.
- Qt policy is redundant across Kvantum/KDE/qt5ct/qt6ct, but observed values are substantially aligned. Do not label it contradictory without runtime proof.
- Noctalia's runtime state intentionally overrides TOML in places. The defect is stale warnings or ignored declared intent, not the existence of mutable state.

Asset provenance

The official default NixOS identity remains blue; rainbow/trans logos are sanctioned variants. The Discourse logo thread is governance context, while official NixOS branding (https://nixos.org/branding/) is normative. The active LightDM wallpaper is byte-identical to an externally sourced asset with a restrictive attribution/no-derivatives license, but local attribution/license metadata is absent. Other copied artwork and theme subsets also lack complete provenance. This is a distribution/provenance issue, not a firmware or boot defect. Record source URL, revision, license, TASL attribution, unmodified or modified status, and retain upstream notices—or replace with owned/permissively licensed assets.

Darwin-specific boundaries

- Apple Silicon: declarations evaluate, but native `darwin-rebuild`, rollback, CoreText/Kitty rendering, TCC, Aqua launch, Spotlight, and LaunchServices are unverified.
- Intel Darwin: retire or freeze on the final supported 26.05 graph with a dated canary and end-of-life plan.
- Two GUI application projection owners overlap on 111 names. This proves duplicate visible ownership, not doubled Nix store bytes. Select one owner and observe LaunchServices/Spotlight before and after on native macOS.
- Standalone identity is fixed to `mei`; ambient overrides conflict with Den's fixed identity. Exact Home Manager target prefixing refutes a feared write to filesystem root, but ambient path construction remains fragile. Use a typed identity record instead.
- Darwin is deliberately Homebrew-free. Do not reintroduce a Homebrew module, casks, or `/opt/homebrew` fallbacks without a new explicit migration decision. Native checks should assert the invariant as well as app behavior.

AI, GPU acceleration, and editor configuration

At the exact root pin, bare Ollama is the CPU/default output. Linux accelerator variants have meaningful platform constraints; CUDA, ROCm, and Vulkan require the matching physical capability and runtime test. Darwin-labelled accelerator aliases can collapse to the ordinary output, so output names are not proof that Metal/GPU execution occurs. The same principle applies to llama.cpp: build flags and package availability are not throughput, offload, memory-fit, or thermal evidence. The supplied llama.cpp guide is a useful secondary tuning reference, not a portable Nix support contract.

Zed is broadly packaged, but its current configuration is standalone-only. No VS Code configuration exists. The supplied dynamic GitHub `path:vscode.nix` search was sampled but exhaustive pagination was rate-limited; exact upstream modules and pinned representative repositories were inspected instead. The claim is therefore about common patterns—package ownership, extension IDs, settings, platform filters—not a census of all GitHub configs.

Shared Emacs is a more immediate portability problem than either editor: its startup uses an HTTP GNU ELPA archive, installs packages on demand, and downloads/evaluates the moving Straight `develop` installer. A clean home can therefore depend on mutable network state on Linux and Darwin. Move package ownership to pinned Nix/content-bound sources, require HTTPS, and test a clean home with the network disabled on both native platforms.

Recommended policy:

- shared: editor selection, settings schema, extension provenance, CPU fallback, model storage location, and opt-in capability interface;
- host-specific: CUDA/ROCm/Vulkan/Metal backend, driver/toolkit version, GPU architecture, memory budget, sandbox/device access, and benchmark evidence;
- reject: global accelerator flags, platform-unsupported extension/package escape hatches, and configuration labels treated as runtime proof.

CI, updates, rollback, and operational debugging

Current evidence surface

The repository exports only three lightweight checks per system, has no GitHub workflow, and leaves six regression entrypoints plus two Disko tests outside the flake check surface. `nix flake check --all-systems --no-build` passed, but it did not realize foreign-platform outputs. A green no-build evaluation is not a native build, VM boot, activation, or physical test.

Proportional gate hierarchy

1. **Gate A—fast deterministic:** clean diff, metadata/show, all-system no-build, warning budget, formatting, host inventory, no `%DISK%`, strict package policy, exact authority checks, exported test inventory.
2. **Gate B—native PR matrix:** x86 Linux, ARM Linux, Apple-Silicon Darwin, and Intel Darwin only if retained on its supported graph. Build checks, every NixOS toplevel, HM activation package, Darwin system, local packages, and smoke-test apps.
3. **Gate C—nightly Linux integration:** exported Disko tests and NixOS VM tests for boot, NetworkManager ownership, Niri/Noctalia, notification/lock owner, portals, OBS resolution, bootstrap negatives, and rollback.
4. **Gate D—release/destructive:** strict known-host behavior, content-bound kexec, disk-manifest mismatches, extra-file/chown order, full reboot/key continuity, install-only recovery, and preserved previous system.
5. **Gate E—physical qualification:** per-host firmware/driver/network/GPU/audio/power/session/rollback evidence. CI cannot certify hardware it never sees.

Use immutable action pins, least privilege, read-only cache access for fork PRs, trusted-branch cache writes, concurrency cancellation, explicit timeouts, and no `continue-on-error` for required jobs. The official Nix GitHub Actions guidance (https://nix.dev/guides/recipes/continuous-integration-github-actions.html) should control over older templates.

Update transaction

Update one trust domain at a time—Nixpkgs/core modules, desktop shell, installer/Disko, overlays, or assets. Record old/new revision and NAR hash, upstream/tag provenance, release notes, advisories, and local fixed-output hashes. Run the gates required by that domain, compare closures, deploy to one canary, test/reboot/rollback, and retain the previous lock/closure/installer/backup for the whole rollback window.

Useful commands answer different questions:

- `nix log <drv>`: earliest derivation failure;
- `nix why-depends <closure> <path>` or `nix-tree`: why a store path is in a closure;
- `definitionsWithLocations`: which Nix module set an option;
- filesystem/inode/root/boot telemetry: why space is exhausted.

Do not begin disk-pressure response with `nix-collect-garbage -d`; first identify filesystems, inodes, roots, generations, result links, running processes, boot payloads, and cache misses.

What the supplied links taught

All 60 source entries were classified; the complete per-URL disposition is in `source-ledger.md`. The durable lessons are:

Official NixOS/Nix/nixpkgs material

The NixOS manual (https://nixos.org/manual/nixos/stable/), Nixpkgs manual (https://nixos.org/manual/nixpkgs/stable/), How Nix Works (https://nixos.org/guides/how-nix-works/), and Nix Pills (https://nixos.org/guides/nix-pills/) establish the core boundary: locked declarative software is not physical hardware, firmware, partition, or mutable-state reproducibility. The Pills remain conceptual/historical; current manuals and exact pinned source control executable guidance. The modern NixOS Wiki is a useful supplement; legacy `nixos.wiki` pages such as Steam require current-source confirmation.

Multi-host and desktop dotfiles

The supplied IogaMaster, eljangus, khyryra, NazariiPalahnii, AdrielVelazquez, QuackHack-McBlindy, kiriwalawren, basnijholt, blkflth, and related Niri repositories converged on useful patterns: explicit named hosts, shared modules plus hardware leaves, semantic and VM tests, Niri fragments by concern, and typed host-specific render/DRM/brightness/GPU values. The most transferable examples were basnijholt's `runNixOSTest` approach, nixflix's unit/VM separation, and explicit host Niri hardware fragments—not their exact package lists or aesthetics. For example, see basnijholt's test projection (https://github.com/basnijholt/dotfiles/blob/a64ef2920fd9ca97410a6ded55e38176c32badcc/configs/nixos/flake.nix#L305-L320), nixflix unit tests (https://github.com/kiriwalawren/nixflix/blob/141cfe004293aad221ab89ddacc4c30c7f6af2cf/tests/unit-tests/default.nix#L17-L46), and Adriel's typed Niri hardware values (https://github.com/AdrielVelazquez/nixos-config/blob/4aef017b2ae10e26a77b6489763c6fa0d603de00/modules/home-manager/niri/default.nix#L27-L67).

Their common anti-patterns are equally instructive: stale install docs, missing tracked hardware files, local/private inputs, public/default credentials, password hashes in the store, hard-coded home/monitor paths, hidden recursive module discovery, sandbox-disabled tests, duplicated attributes, dead compositor modules, and imperative dotfile ownership. Unixporn/rice sources are interaction and visual catalogs, not correctness or security foundations.

Deployment, image, WSL, server, and Dendritic material

The SCALE home-server talk, Dustin Lyons' configuration, nixos-images, Determinate's ISO, Hyper-V reports, WSL starters, Icicle, Den, and Dendritic material support one tree with explicit outputs and artifact choice by target: ISO for local install, authenticated nixos-anywhere for reachable systems, pinned kexec only when necessary, provider images for clouds/VMs, `.wsl` artifacts for WSL, and nix-darwin for macOS. nixos-images (https://github.com/nix-community/nixos-images) is valuable for installer/recovery artifacts but still has platform, RAM, Secure Boot, firmware, and network-preservation limits. Determinate's ISO (https://github.com/DeterminateSystems/nixos-iso) changes the Nix distribution/tooling and is not a neutral drop-in. Icicle is an installer product, not fleet configuration management.

The durable Dendritic idea is feature-first class-specific composition; its own FAQ warns against vocabulary fanaticism. The current Den design can be retained if every entity/class is tested. Do not add another abstraction merely because popular configs do.

CI, performance, and operations material

The supplied CI template is stale and Ubuntu-only for a nominally multi-system flake. Fenix demonstrates stronger Linux/macOS/update matrices, but floating or failure-tolerant workflow choices should not be copied blindly. r-ryantm PR activity and update bots are leads, not proof. nix-tree is useful for closure attribution, while module definitions and derivation logs answer different questions. The devenv performance article depends on workload, hardware, pin, and an open Nixpkgs optimization; benchmark after correctness gates rather than adopting headline numbers as readiness evidence.

Videos, Reddit, community guides, and historical posts

The videos and community guides are useful for motivation and workflow ideas, not final authority. The Ioga/nixos-anywhere video omits some current destructive trust and platform boundaries; modern nix.dev provisioning supersedes it. The No Boilerplate video usefully explains declarative generations but overstates instant reproduction and the impossibility of unbootable state. The WSL video reinforces WSL as a separate platform but uses an older workflow. Funloop's 2015 Arch/Nix article is historically useful for gradual adoption, not current `nix-env`/channel instructions. Reddit opinions on stability and migration are contradictory and contextual. The tsawyer guide has useful stable-ID/Disko ideas but contains unsafe/stale copy-paste details, including invalid flake syntax, moving inputs, hash exposure, and host-specific destructive assumptions.

Niri, Noctalia, branding, editors, and local AI

The supplied Noctalia v4 issue is obsolete for the pinned v5 rewrite. Exact Noctalia/Niri source and generated artifacts, not the old issue, control the lock/notification/portal findings. `niri-flake` is comparative only because the repository already uses upstream Niri packaging. Official branding, not a Discourse debate, controls logo/color policy. GitHub VS Code code search was partially rate-limited and therefore cannot support an ecosystem census. The llama.cpp guide offers tuning leads, but physical backend benchmarks control support claims.

Rejected, narrowed, or obsolete patterns

Do not reintroduce these claims without new evidence:

Rejected, narrowed, or obsolete claims from the source corpus

Claim/pattern	Final disposition
One closure can never span diverse hardware	**Refuted as absolute.** It can span a bounded compatibility class; it cannot mean every boot/storage/firmware/driver class.
Every machine needs Facter or nixos-hardware to boot	**Refuted as prerequisite.** They improve assurance and exception routing; named leaves bind support/destructive choices.
Missing SSID explains the absent WLAN netdev	**Refuted.** Enumerate controller, driver, firmware, rfkill, and kernel first.
NetworkManager and `networking.wireless` are competing managers here	**Refuted at the exact pin.** wpa_supplicant is intentionally controlled by NetworkManager.
Facter+NetworkManager is a current defect	**Narrowed.** It is a demonstrated prospective collision only after importing Facter without disabling detected DHCP.
`%DISK%` itself fails open	**Refuted.** Literal placeholder fails closed; wrong valid replacement remains destructive.
Red installTest proves the real install fails	**Refuted.** It proves a non-self-contained fixture; isolated disk/boot passes; real transport remains unverified.
100 MiB cannot boot one generation	**Refuted.** It boots and fits two unique current payloads, but has unsafe update/rollback margin.
2 GiB safely holds 42 current payloads with 20% margin	**Refuted.** Exact occupancy is 82.98%; use dynamic bytes or ≥3 GiB.
Integrated NixOS has duplicate Noctalia units	**Refuted.** It has one; standalone HM is a separate output.
All three notification daemons certainly run simultaneously	**Narrowed.** Multiple activatable candidates exist; the winner is ordering-dependent.
X11 units are proven continuously active under Niri	**Narrowed.** They are scheduled; they may run, fail, or retry.
Runtime Noctalia state is inherently defective	**Refuted.** It is upstream policy; stale keys and unwanted ownership are the defects.
Portal routing proves full RemoteDesktop control	**Narrowed.** ScreenCast is present; Niri lacks Mutter's RemoteDesktop input-injection interface.
GTK and Qt both have contradictory values	**Narrowed.** GTK conflicts; Qt is redundant but substantially aligned.
Kitty/Fira theming is absent on macOS	**Refuted statically.** It reaches all six outputs; native rendering is deferred.
Duplicate Darwin app projection doubles store bytes	**Narrowed.** Visible ownership overlaps; store paths can deduplicate.
Current agenix exposes plaintext	**Refuted.** No age secrets are active; future identity/runtime design remains necessary.
`allowBroken` currently hides a selected broken package	**Not demonstrated.** It is latent; `allowUnsupportedSystem` has executed counterexamples.

Claim/pattern	Final disposition
Missing microcode causes generic boot failure	**Refuted.** It is security/errata freshness debt.
Git checkout on USB is an offline installer	**Refuted.** It lacks realized closure, inputs, installer tooling, firmware, and a cold-store proof.
“Always rollback” or “flakes make unbootable states impossible”	**Rejected.** GC, boot capacity, firmware, partitions, external state, and mutable data remain outside that slogan.
Old channels/ `nix-env` , moving `latest` kexec/inputs, root `curl | sh` , and unverified community copy-paste commands	**Obsolete or unsafe.** Use exact pins, current official interfaces, and target-specific verification.

Exact commands and verification gates

These commands are the required *shape* of the next implementation's evidence. Attribute names should become named hosts rather than architecture labels.

Fast repository gate

```
git diff --check
nix flake metadata --json > evidence/flake-metadata.json
nix flake show --json > evidence/flake-show.json
nix flake check --all-systems --no-build --show-trace \
  2>evidence/flake-check-warnings.log
nix fmt -- --check
```

Also assert: no `%DISK%` in installable host outputs; global broken/unsupported escapes are false; every test entrypoint is exported; every host has architecture, boot mode, stable disk, hardware evidence, and network fallback; exactly one notification and lock authority; requested theme assets exist.

Native builds

```
# Linux, on each native architecture
nix flake check -L
nix build --no-link ".#nixosConfigurations.${HOST}.config.system.build.toplevel"
nix build --no-link ".#homeConfigurations.${HOME_NAME}.activationPackage"

# Darwin, on matching native macOS hardware
nix build --no-link ".#darwinConfigurations.${SYSTEM}.system"

# Review size and changes before activation
nix path-info -Sh "$NEW_SYSTEM"
nix store diff-closures "$OLD_SYSTEM" "$NEW_SYSTEM"
```

Disko and VM integration

```
nix build -L --no-link \
  ".#nixosConfigurations.${HOST}.config.system.build.installTest"
```

The exported VM suite must cover dummy-secret missing/wrong/correct cases, NetworkManager-only ownership, one notification/lock owner, Niri software login, portal names, exact Noctalia lock command, OBS command resolution, rollback boot, ESP capacity threshold, and wrong disk/mount failure before writes.

Hardware and Wi-Fi intake on the target

Run at a local console and keep serial/MAC-bearing raw output private:

```
uname -m
cat /etc/os-release
cat /sys/firmware/efi/fw_platform_size 2>/dev/null || true
ip -br link
find /sys/class/net -maxdepth 1 -mindepth 1 -printf '%f\n'
lsblk -e7 -o NAME,PATH,TYPE,SIZE,MODEL,SERIAL,WWN,TRAN,RM,FSTYPE,MOUNTPOINTS
find /dev/disk/by-id -maxdepth 1 -type l -printf '%p -> %l\n'
lspci -nnk
lsusb -t
rfkill list
iw dev
nmcli radio all
nmcli -f DEVICE,TYPE,STATE,CONNECTION device
systemctl is-active NetworkManager wpa_supplicant dhcpcd
bootctl status
journalctl -b -k --no-pager
cat /sys/power/mem_sleep
```

Review the kernel journal for firmware load failures, microcode, WLAN drivers, GPU faults, storage resets, audio/SOF, rfkill, suspend, and IOMMU—not merely whether services are green.

Installer, SSH, and disk identity

Before any destructive phase:

```
# From a trusted console: record the installer's public fingerprint
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

# Final daemon policy after install
sudo sshd -T | grep -E \
  '^(permitrootlogin|passwordauthentication|kbdinteractiveauthentication) '

# Disk and mount re-check immediately before Disko
lsblk -e7 -o NAME,PATH,TYPE,SIZE,MODEL,SERIAL,WWN,TRAN,RM,FSTYPE,MOUNTPOINTS
findmnt -R
```

Negative controls are mandatory: a wrong host fingerprint, kexec digest, disk serial/model/size/by-id, architecture, boot mode, mount source, bootstrap fixture, and Wi-Fi keyfile mode must stop before their protected action.

SSH, password, and secret closure

```
sudo sshd -T
passwd -S mei
sudo chage -l mei
systemctl --failed --no-pager
systemctl --user --failed --no-pager
```

Test normal-user key login and sudo positively; test root key, password, and keyboard-interactive SSH negatively. Secret scans must emit only rule/path/ref metadata, never secret bytes. When agenix is activated, prove plaintext absent from closure strings, logs, argv, environment, and world-readable paths, while the runtime file has expected owner/mode and cleanup.

Disk/GC/rollback diagnostics

```
df -h /nix /boot
df -i /nix /boot
sudo nix-env --profile /nix/var/nix/profiles/system --list-generations
nix profile history
nix-store --gc --print-dead > evidence/gc-dead.txt
bootctl status
find /boot/EFI/Linux /boot/loader/entries -maxdepth 1 -type f \
  -printf '%s %p\n' 2>/dev/null
```

Do not delete until the current and previous known-good closure, boot entry, lock, backup, and recovery path are identified.

Offline kit

```
host=concrete-host
store="file:///media/$USER/ventoy/nix-offline-store"
system=$(nix build --no-link --print-out-paths \
  ".#nixosConfigurations.$host.config.system.build.toplevel")
nix flake archive --to "$store" .
nix copy --to "$store" "$system"
nix path-info --store "$store" -r "$system" >/dev/null
```

Also copy the exact installer tooling closure. Prove completeness from a fresh or empty test store with network disabled; `--offline` against a warm normal store is not evidence.

Physical host enrollment and release checklist

No row may be marked pass from static or VM evidence alone.

Before installation

- [ ] Name the host; record repository commit, intended output, architecture, manufacturer/model, BIOS/UEFI revision, and intended support class.
- [ ] Boot the verified installer in the intended firmware mode; record UEFI bitness, Secure Boot state, NVRAM ability, and checksum provenance.
- [ ] Collect Facter/generated hardware config plus independent `lspci -nnk`, `lsusb`, `lsblk`, by-id, rfkill, GPU, storage, and network evidence.
- [ ] Sanitize persistent identifiers before any report enters Git/Nix; retain raw evidence privately.
- [ ] Review an exact `nixos-hardware` model profile if one exists; do not import a merely similar model.
- [ ] Bind stable disk ID, resolved node, model, serial/WWN, size, transport, removable flag, descendants, boot mode, and wipe intent. Exclude installer media and mounted valuable disks.
- [ ] Confirm root/storage/input initrd drivers, CPU microcode, firmware/regdb, GPU policy, and any proprietary exception.
- [ ] Test Ethernet/USB-Ethernet/tether fallback in both installer and intended closure. Decide local `nmtui` or authenticated encrypted Wi-Fi handoff.
- [ ] Prove wrong host key, artifact digest, disk identity, and secret fixture fail before upload, execution, or write.

First boot and hardware

- [ ] Local TTY login works before relying on graphics or networking.
- [ ] Expected PCI/USB devices bind expected drivers; required firmware loads without unresolved errors; appropriate CPU microcode is active.
- [ ] Internal storage and ESP have expected filesystem, controller, size, and free-byte headroom; current and previous generation both boot.

- [ ] Wi-Fi netdev exists; regulatory domain, rfkill, scan, WPA2/WPA3, association, DHCP, DNS, reboot autoconnect, and fallback network work.
- [ ] Bluetooth, audio, microphone, touchpad/keyboard, brightness, camera, fingerprint/dock/WWAN where claimed all work.
- [ ] GPU acceleration, internal/external displays, video acceleration, and unplug/replug behavior work.
- [ ] Suspend/resume is repeated; Wi-Fi, audio, input, display, Bluetooth, and dock recover. Test hibernation only with an explicit swap/resume design.
- [ ] For external displays, `/dev/i2c-*`, group/uaccess permissions, `ddcutil detect`, and Noctalia DDC control work. NixOS owns I2C declaratively; standalone uses the attended helper and never performs surprise sudo during Home Manager activation.
- [ ] Timezone and keyboard layout come from the enrolled host/user/location policy rather than the shared workstation baseline.

Niri and presentation

- [ ] Cold login has no failed system/user units and exactly one notification owner, one lock authority, one wallpaper owner, and one integrated Noctalia unit.
- [ ] Manual/idle lock covers every output; unlock, suspend/resume, lid, and output hotplug do not expose an unlocked surface.
- [ ] Screenshot and screen/window capture work through real portal clients, with expected consent; file chooser/settings/Secret work. Do not claim RemoteDesktop input injection unless separately implemented.
- [ ] OBS launches if advertised and captures via PipeWire.
- [ ] Kitty font/palette, GTK 3/4, Qt, cursor, icons, wallpaper, and Noctalia render correctly; capture evidence at representative scale/output states.
- [ ] On macOS, native activation/rollback, CoreText font registration, Kitty, Aqua apps, TCC, Spotlight, and LaunchServices are tested separately.

Security and lifecycle

- [ ] `sshd -T` matches policy; user key and sudo succeed; root/password/keyboard-interactive SSH fail; a tested offline recovery path remains.
- [ ] Bootstrap password is changed or deliberately locked; verifier copy is gone. No hash or password is printed.
- [ ] Provider credential has been rotated/revoked and activity reviewed; current-tree/full-history scanners are clean without reproducing values.

- [ ] If agenix is active, per-host recipient isolation, runtime mode/owner, closure/log/argv/environment absence, rotation, and cleanup all pass.
- [ ] `nixos-rebuild test`, health check, `boot`, reboot, previous generation, `switch`, GC rehearsal, and mutable-data restore all pass.
- [ ] Record commit, closure, kernel, firmware/BIOS, PCI/USB IDs, test results, failures, exceptions, date, and expiry triggers. One host does not qualify a different controller/GPU/firmware class.

Evidence, inference, and what remains unverified

Verified directly

- Repository option/path/pin/ownership facts at `e9f7818`.
- Root Nixpkgs revision `d407951…`; an earlier transitive/helper `59e696…` identification is superseded for package claims.
- Exact generated units/configs/packages, all six evaluation paths, and realized x86 closure contents.
- Exact Noctalia command behavior, OBS command lookup, notification activators, theme asset presence/absence, platform availability counterexamples, SSH option precedence, and `sync-secrets` failure.
- Synthetic Facter/NetworkManager interaction, exact FAT allocation, systemd-boot ENOSPC control flow, age boundary primitive, signed-manifest controls, and isolated OVMF/Disko boot.
- All 60 supplied links classified; 15 independent base lanes, two expansion waves, and the closing repository-context audit completed; 141 observations and 58 canonical claims are registered; credential value absent from research artifacts.

Inferred or modeled

- The exact x86 closure is *plausible* for the bounded mainstream UEFI-PC class.
- The AArch64 output is at most a plausible generic SystemReady/UEFI template.
- Current GC/ESP policy cannot guarantee rollback even though the exact fault behavior is modeled from source and synthetic injection.
- A full offline kit requires all closure/input/tooling content even though a physical disconnected install was not run.

Explicitly not verified

- The real laptop's missing WLAN controller/driver/firmware/rfkill cause or a successful physical Wi-Fi association.
- Unchanged closure qualification across Intel and AMD PCs with varied NIC/GPU/storage; NVIDIA/hybrid, special firmware, Surface/T2, docks, cameras, and fingerprint devices.
- Any physical AArch64 boot; Secure Boot, BIOS, 32-bit EFI, ARM board boot, or Apple Silicon NixOS.
- A real sacrificial destructive install, actual disk identity, strict SSH continuity across pre-installer/installer/final stages, or power-cut recovery.
- Cold installed Niri login, multi-output locking, portal consent, suspend, rendering, or wallpaper behavior.
- Native Apple Silicon or Intel macOS build/activation/CoreText/LaunchServices.
- Disconnected offline install from Ventoy; external provider revocation/account review; old credential validity (intentionally not tested).

Repository/source evidence may establish declarations. Evaluation establishes selected artifacts. Synthetic tests establish their modeled contract. OVMF establishes only that VM path. None of those layers may be relabeled as physical firmware, driver, radio, power, display, or recovery proof.

Implementation sequence and stop boundary

1. **External incident first:** rotate/revoke the provider credential and review activity without testing or sharing the old value.
2. **Repository P0 security:** remove current secret copies, add redacted scans, harden installed SSH/password lifecycle, and select a strict installer path.
3. **Install model:** introduce named host/storage/hardware leaves, disk manifest validation, content-bound installer artifact, and attended ISO default.
4. **Fix existing red evidence:** self-contained Disko fixture and exported negative controls.
5. **Desktop authority:** correct Noctalia lock, remove X11 lock stack, choose one notification and wallpaper owner, resolve OBS and stale schema.
6. **Platform honesty:** remove global unsupported/broken escapes; retire or isolate Intel Darwin; fix typed identity and Darwin app ownership.
7. **Hardware/network:** enroll the laptop, enable correct microcode, route exact firmware/driver exceptions, separate NetworkManager role, disable prospective Facter DHCP, and establish local/encrypted Wi-Fi handoff plus fallback.

8. **Boot/rollback:** resize/restructure ESP/XBOOTLDR, add byte preflight and last-known-good invariant, align GC with rollback and backup.
9. **Theme and apps:** choose GTK/Qt/font/cursor/wallpaper authorities, preserve asset provenance, retain Kitty across all outputs, and run native visual tests.
10. **Compatibility and developer startup:** preserve public architecture aliases, Bash/Nushell parity, Niri-only login, Homebrew-free Darwin, and backup-safe standalone adoption; pin Emacs and the standalone installer; route timezone/XKB by host and physically qualify DDC/I2C.
11. **Evidence system:** export semantic/native/VM/installer gates, run release canary, then execute and record physical enrollment on each claimed host.

Stop boundary: this report is research and verification, not an authorization to perform destructive installation, rewrite public Git history, rotate external credentials on the user's behalf, or claim physical readiness. Implementation should stop before any disk write or external irreversible action unless the named-host manifest, authenticated artifact/host path, backup/recovery plan, and attended confirmation are present. The universal-readiness claim remains false until every retained support class has current physical acceptance evidence.

Artifact index

- `verified-claims.md` — canonical P0/P1/P2, positives, deferred evidence, six-output matrix, support classes, contradiction register.
- `claim-graph.md` — intent dependencies, canonical statuses, counterevidence, validity, and refuted/narrowed nodes.
- `intent-diff.md` — requested outcomes versus observed reality.
- `cause-disappearance.md` — rejected causal theories and their replacements.
- `observation-manifest.md` — O1–O141 evidence ledger.
- `source-ledger.md` — disposition of all 60 supplied links.
- `closing-context-audit.md` — final tracked-tree and Lore-history constraints covering Emacs, installer trust, migration, Darwin, compatibility interfaces, DDC/I2C, timezone, and keyboard policy.
- `source-snapshots/MANIFEST.sha256` — content hashes for the exact local source excerpts linked by this portable report; snapshots are from production revision `e9f7818`.
- `verify-final.md` — final exact-tree command results and intentional red/negative evidence.
- `verification-economics.md` — cheap, native, synthetic, destructive, and physical proof boundaries.

- `wave-2-host-intake-wifi.md` — host enrollment, Facter privacy/DHCP, Wi-Fi handoff, offline closure, and physical gates.
- `wave-2-esp-vm-capacity.md` — exact Disko/OVMF and FAT capacity experiment.
- `wave-2-ssh-kexec-trust.md` — SSH option order, kexec identity, key continuity, manifest harness.
- `wave-2-secrets-remediation.md` — redacted incident, agenix/runtime design, SSH/bootstrap lifecycle, sync bug.
- `wave-2-session-runtime.md` — exact Niri, Noctalia, D-Bus, portals, OBS, and theme-generated artifact checks.
- `wave-2-ci-lifecycle.md` — gate hierarchy, native matrix, update transaction, GC/rollback policy.
- `wave-3-hardware-skeptic.md`, `wave-3-network-trust-skeptic.md`, `wave-3-boot-disk-skeptic.md`, `wave-3-session-skeptic.md`, `wave-3-cross-platform-skeptic.md`, and `wave-3-ci-supply-skeptic.md` — adversarial corrections that control over earlier, broader wording.

Evidence snapshot at commit `e9f78180748f1feb428ffb20f9d932c5d9918a48`. Physical-machine and external-account actions remain explicitly deferred unless marked verified.